



Bangladesh University of Engineering and Technology

Department of Electrical and Electronics Engineering

Course No: EEE212

Course Name: Numerical Techniques Laboratory

Project

Submitted By

Name: Md. Symria Raihan **ID:** 2006133

Name: S. M. Razibur Rahman **ID:** 2006134

Level: 2 **Term:** 1 **Section:** C1

Date of Submission: 28-02-2023

Forwarding Letter

February 28, 2023

To
Dr. Hafiz Imtiaz
Sujoy Mondal

Department of EEE
Bangladesh University of Engineering and Technology, Dhaka-1000

Subject: Application for report submission on "**Analysis of Continuous Signals.**"

Dear Teachers,

It gives us great pleasure and the chance to submit the report "**Analysis of Continuous Signal**" and complete the specified project. Working on this report has been an honor, and we want to thank you from the bottom of our hearts for enabling us to complete this report and the project in EEE212.

We did our best to make the project more approachable and tried to write an informative piece that would make it relatively simple to understand the continuous signal. While putting the project into action and writing the report, we made every effort to be as accurate and objective as possible. Our group would like to express its profound thanks for your unwavering support, ongoing assistance, and sage advice as we proceed with this fascinating issue.

Sincerely Yours

Md. Symria Raihan 2006133

S. M. Razibur Rahman 2006134

Table of Contents

Table of Contents iii
1. Introduction 1
2. Theoretical Background 2
2.1 Continuous Signal: 2
Amplitude Modulation 3
2.2 Basic Operation 1: 3
2.2 Basic Operation 2: 5
2.3 Determination of Fourier series 6
2.4 Fourier Transform: 8
2.5 Amplitude Modulation: 8
3. Graphical User Interface (GUI) 9
3.2. User Manual 14
4.Main Code 14
5.Test Cases 27
6.Applications 36
7.Shortcomings 37
8. Conclusion 38

1. Introduction

This report aims to present the results of analyzing a given signal using various mathematical operations such as addition, multiplication, convolution, scaling, shifting, folding, Fourier series, Fourier transform, and amplitude modulation. These operations are commonly used in signal processing and are essential for understanding the behavior of signals in different domains.

The signal under consideration is a continuous-time signal, denoted by s(t), finite in duration. We have performed various operations on this signal to understand its properties in different domains.

Firstly, we have performed addition and multiplication operations on the signal. Addition involves adding the values of the signal at each point in time, while multiplication involves multiplying the values of the signal at each point in time. These operations combine signals and can be used in various applications such as audio mixing.

We have also performed convolution, a mathematical operation used to find a system's response to an input signal. In this case, we have convolved the signal with itself to find the output signal. Convolution is widely used in signal processing and is an essential tool for understanding the behavior of systems.

Scaling and shifting are operations that involve changing the amplitude and time of a signal, respectively. Folding involves reflecting a signal about a specific point in time. We have performed these operations on the signal to understand their effects on the signal.

Fourier series and Fourier transform are powerful tools for analyzing signals in the frequency domain. We have applied these tools to the signal to obtain its frequency domain representation. This helps understand the signal’s spectral content and is helpful in applications such as audio compression.

Finally, we have performed amplitude modulation, a technique used in communication systems for transmitting signals over a long distance. Amplitude modulation involves multiplying a high-frequency carrier signal with the baseband signal, and we have applied this technique to the given signal.

In conclusion, this report presents the results of various mathematical operations performed on a given signal. These operations are fundamental to signal processing and essential for

understanding signals' behavior in different domains. The results of this analysis can be applied in various applications such as audio and communication systems.

2. Theoretical Background

2.1 Continuous Signal:

Continuous signals exist in the continuous domain of time and amplitude. In other words, they can take on any value at any point, and the values can vary continuously over time. Continuous signals can be represented mathematically as functions of time, where the time variable is a continuous variable, often denoted by the letter t. An analog audio signal is one of the most common examples of a continuous signal. Audio signals are continuous in both the time and amplitude domains and can take on any value within a certain range of amplitudes. Other example of continuous signals includes analog video signals, electromagnetic waves, and various measurements such as temperature, pressure, and velocity. Continuous signals find application in a wide range of fields. In Telecommunications, they are used for transmitting and receiving information over long distances. In audio engineering, they are used for recording, mixing, and producing music. In physics and engineering, they are used for analyzing and modeling physical phenomena such as oscillations, vibrations, and waves.

One of the main advantages of continuous signals is their ability to capture a wide range of information accurately. Because continuous signals can take on any value, they can represent complex signals with high fidelity. This makes them particularly useful in applications requiring precise measurements and high-quality data.

However, continuous signals can also be subject to noise and distortion, particularly when transmitted over long distances or converted to digital formats. As a result, signal processing techniques such as filtering, modulation, and demodulation are often used to improve the quality of continuous signals in various applications.

We could perform different types of operation for a continuous signal. We have divided those operation into five parts. Such as-

1. Basic Operation 1
2. Basic Operation 2

3. Implementation of Fourier series
   3.1 Trigonometric Fourier Series
   3.2 Exponential Fourier Series
4. Fourier Transform

Amplitude Modulation

2.2 Basic Operation 1:

5.

In Basic Operation 1, The operations requiring two signals are performed. Such as- Addition, Convolution and Multiplication.

Addition:

Adding two continuous signals involves combining the amplitude of the two signals at each point in time. The resulting signal is the sum of the individual signals, each contributing to the overall amplitude at each point. This operation is often used in signal processing applications to combine two signals, such as when mixing audio signals or transmitting multiple signals over a single channel. The addition of signals can also be used to extract information from a noisy signal by adding a clean reference signal to the noisy signal and subtracting the resulting signal from the original noisy signal. This technique, known as subtraction or cancellation, can help remove unwanted noise from the signal and improve its quality. Adding two continuous signals is a fundamental operation in signal processing and is important in many applications.

Convolution:

Convolution is a mathematical operation commonly used in signal processing to combine two signals in the time domain. It involves multiplying two functions, one of which is usually a time-varying signal, and integrating the product over time. The result of the convolution operation is a new signal that representing signal is affected by the other as it is shifted in time.

One common application of convolution is in filtering. A filter is a system that modifies a signal by removing or enhancing certain frequency components. Convolution can apply a filter to a signal by multiplying the signal with a filter kernel in the time domain. The resulting signal is then integrated over time to produce the filtered output signal.

Another application of convolution is in correlation. Correlation is a technique used to compare two signals to determine their similarity. Convolution can be used to correlate by sliding one signal over another and multiplying their overlapping values. The resulting signal represents the correlation between the two signals, with high values indicating a substantial similarity.

Convolution is also used in image processing, where it is used to apply a spatial filter to an image. In this case, the filter kernel is a matrix of values convolved with the image pixel values to produce a new image that has been smoothed, sharpened, or otherwise modified. Overall, convolution is a fundamental operation in signal processing and has many important applications in various fields, including audio processing, image processing, and communication systems.

$$(f * g)(t) = \int f(\tau)\, g(t - \tau)\, d\tau$$

Fig: Basic formula of Convolution

Multiplication:

Multiplying two continuous signals involves modulating one signal with another signal. The resulting signal is the product of the individual signals and represents the combined frequency components of the two signals. This operation is often used in signal processing applications such as amplitude modulation, frequency modulation, and phase modulation. In amplitude modulation, a carrier signal is multiplied by a modulating signal, which changes the amplitude of the carrier signal. The resulting signal contains the original carrier signal frequency components and two additional frequency components that represent the sum and difference of the carrier and modulating signal frequencies.

In frequency modulation, a carrier signal is multiplied by a modulating signal, which changes the frequency of the carrier signal. The resulting signal contains frequency components proportional to the modulating signal's amplitude. In phase modulation, a carrier signal is multiplied by a modulating signal, which changes the phase of the carrier signal. The resulting signal contains phase shifts that are proportional to the modulating signal's

amplitude of two signals can also be used to perform spectral analysis by using a signal with a known frequency spectrum to "probe" another signal. This technique, called cross-correlation, is used to identify similarities between two signals by multiplying their frequency.

2.2 Basic Operation 2:

In Basic Operation 2, The operations requiring only one signal are performed. Such as – Folding, Shifting and Scaling

Folding:

Folding, also known as time-reversal, is a mathematical operation used in signal processing to reflect a signal around a point in time. This is done by reversing the order of the samples in the signal and flipping the time axis. The result of this operation is a new signal that is the mirror image of the original signal.

Folding has many applications in signal processing. One example is in digital signal processing where it is used to reduce the effects of aliasing. Aliasing occurs when a signal is sampled at a rate that is too low to capture all of its frequency components, resulting in distortion of the signal. By folding the signal, its frequency components are reflected around the sampling rate, and the ones that were previously above the Nyquist frequency (half the sampling rate) are now below it. This allows the signal to be reconstructed correctly without distortion. Folding can also be used in system analysis to determine the response of a linear time-invariant system to a signal. By folding the impulse response of the system, the system's frequency response can be obtained using Fourier analysis.

Folding is a useful operation in signal processing that has applications in various areas such as digital signal processing and system analysis. By reversing the time axis of a signal, folding can be used to reduce the effects of aliasing, determine the frequency response of a system, and more.

Shifting:

Shifting, also known as time shifting or time delay, is a fundamental operation in signal processing that involves changing the time reference of a signal. This is done by adding or subtracting a constant value from the time axis of the signal. The result is a new signal identical to the original signal, but with a time shift. Shifting has many applications in signal

processing. One example is in signal alignment, where two or more signals are shifted in time to align their peaks or other features. This is often done in audio and image processing applications to correct for delays caused by equipment or other factors.

Shifting can also be used in signal filtering to correct for phase distortion caused by the filter. By shifting the signal in time, the filter's phase response can be compensated for, resulting in a clearer and more accurate signal. In addition, shifting is used in time-series analysis to study the relationship between two or more signals. Shifting one signal in time and correlating it with another signal can determine the degree of similarity between the two signals at different time lags.

Scaling:

Time scaling, also known as time compression or expansion, is an operation in signal processing that involves changing the duration of a signal by a constant factor. This is done by stretching or shrinking the time axis of the signal, while preserving its shape and frequency content. Time scaling has many applications in signal processing. One example is in speech processing, where it is used to adjust the speaking rate of a recorded speech signal. In this case, the time scaling factor is typically less than one to slow the speaking rate or greater than one to speed it up. Time scaling can also be used in music processing to adjust the tempo of a musical signal. By stretching or shrinking the time axis of a music signal, its tempo can be slowed down or sped up while preserving its pitch and other musical features.

Again, time scaling is used in time-series analysis to study the relationship between two or more signals that have different sampling rates. By time-scaling one signal to match the sampling rate of another signal, the two signals can be compared and analyzed together. Overall, time scaling is a crucial signal processing operation with many uses, including time-series analysis, voice processing, and music processing. Time scaling can be used to produce desired speech speeds, tempos, or sampling rates for processing and analysis by modifying the length of a signal.

2.3 Determination of Fourier series

According to Baron Jean Baptiste Joseph Fourier, any periodic signal f(t) with angular frequency ω can be expressed as,

$$f(t) = a_0 + a_1 \cos \omega t + a_2 \cos 2\omega t + \cdots + a_n \cos n\omega t + \cdots + b_1 \sin \omega t + b_2 \sin 2\omega t + \cdots + b_n \sin n\omega t + \cdots$$

2.3.1 Trigonometric Fourier Series:

The trigonometric Fourier series of a periodic signal consists of a DC component (a constant term) and a series of harmonics, each of which is a sine or cosine function with a frequency that is an integer multiple of the fundamental frequency of the signal. The coefficients of these harmonics represent the amplitudes and phases of the different frequency components of the signal.

The **coefficients** can be determined by

$$a_0 = \frac{1}{T}\int_{t_1}^{t_1+T} f(t)\,dt$$

$$a_n = \frac{2}{T}\int_{t_1}^{t_1+T} f(t)\cos n\omega t\,dt$$

$$b_n = \frac{2}{T}\int_{t_1}^{t_1+T} f(t)\sin n\omega t\,dt$$

Exponential Fourier Series:

The exponential Fourier series of a periodic signal consists of a DC component (a constant term) and a series of harmonics, each of which is a complex exponential function with a frequency that is an integer multiple of the fundamental frequency of the signal. The coefficients of these harmonics represent the amplitudes and phases of the different frequency components of the signal.

The signal f(t) can be expressed as:

$$f(t) = a_0 + c_1 \sin(\omega t + \theta_1) + c_2 \sin(2\omega t + \theta_2) + \cdots$$

Where,

$$c_n = \sqrt{a_n^2 + b_n^2}$$

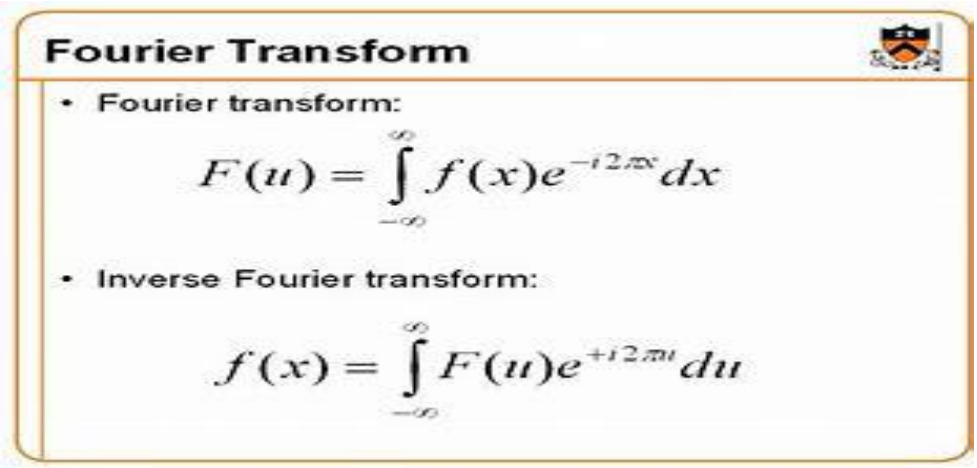
$$\theta = \tan^{-1}\left(\frac{a_n}{b_n}\right)$$

2.4 Fourier Transform:

The Fourier transform converts a time-domain signal into its frequency-domain equivalent. It is a mathematical operation used in signal processing. It bears the name of Joseph Fourier, a French mathematician who, in the early 19th century, created the idea of Fourier series.

A continuous-time signal's Fourier transform is a complex-valued function that sums complex exponential functions with various frequencies to describe the signal. A discrete-time signal's Fourier transform is likewise a complex-valued function, but it has a distinct meaning that considers the signal's discreteness.

Many signal processing techniques use the Fourier transform, such as the synthesis, analysis, and design of filters and other signal processing methods. It is utilized in many industries, including control systems engineering, audio processing, and telecommunications. One of its key characteristics is the Fourier transform's capacity to concurrently represent a signal in the time and frequency domains. This characteristic, sometimes called the duality principle, is a key finding in signal processing and mathematics.



2.5 Amplitude Modulation:

Amplitude modulation (AM) is a technique used in analog communication systems to transmit information over a carrier wave by varying the amplitude of the carrier wave in accordance with the modulating signal.

In AM, the modulating signal is typically a low-frequency audio signal that carries the desired information to be transmitted, such as speech or music. The carrier wave is a high-frequency sine wave that is modulated by the modulating signal to produce an AM signal.

The AM signal can be expressed mathematically as a product of the carrier wave and the modulating signal. The resulting waveform consists of two sidebands, located above and below the carrier frequency, and a residual carrier component.

AM has many applications in radio broadcasting and other communication systems, including in AM radio, shortwave radio, and television broadcasting. It is also used in some wireless communication systems, such as amplitude-shift keying (ASK) and on-off keying (OOK).

The equation of Amplitude Modulated wave will be

$$s(t)=[A_c+A_m\cos(2\pi f_m t)]\cos(2\pi f_c t)$$

Where,

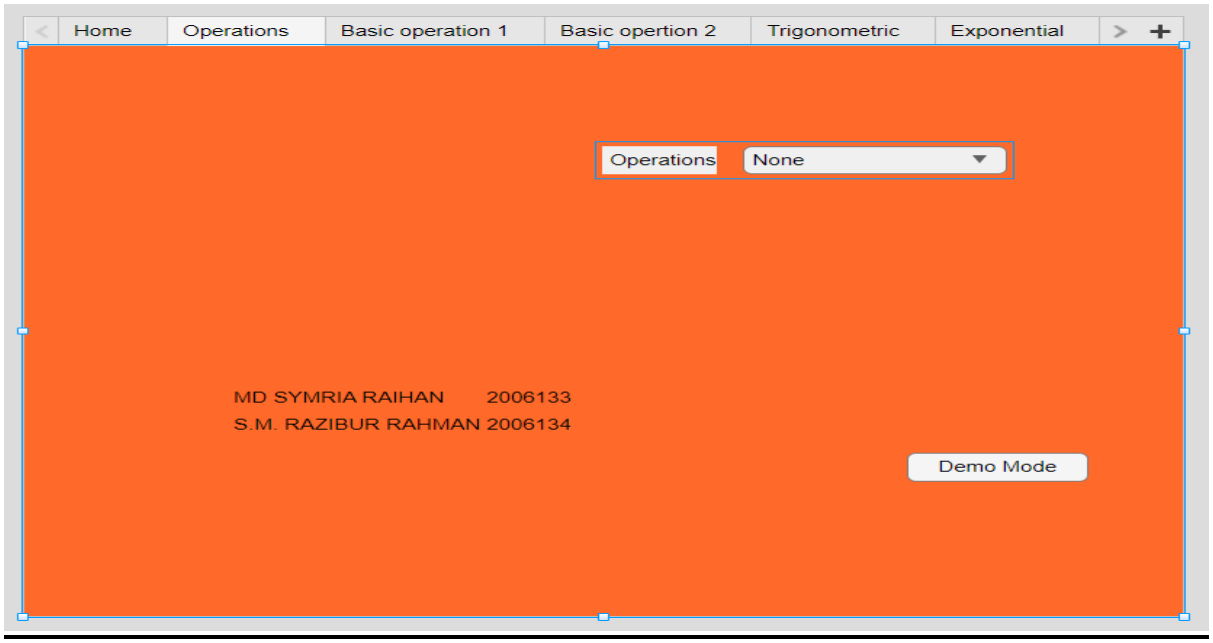
A_m and A_c are the amplitude of the modulating signal and the carrier signal respectively.
f_m and f_c are the frequency of the modulating signal and the carrier signal respectively.

3. Graphical User Interface (GUI)

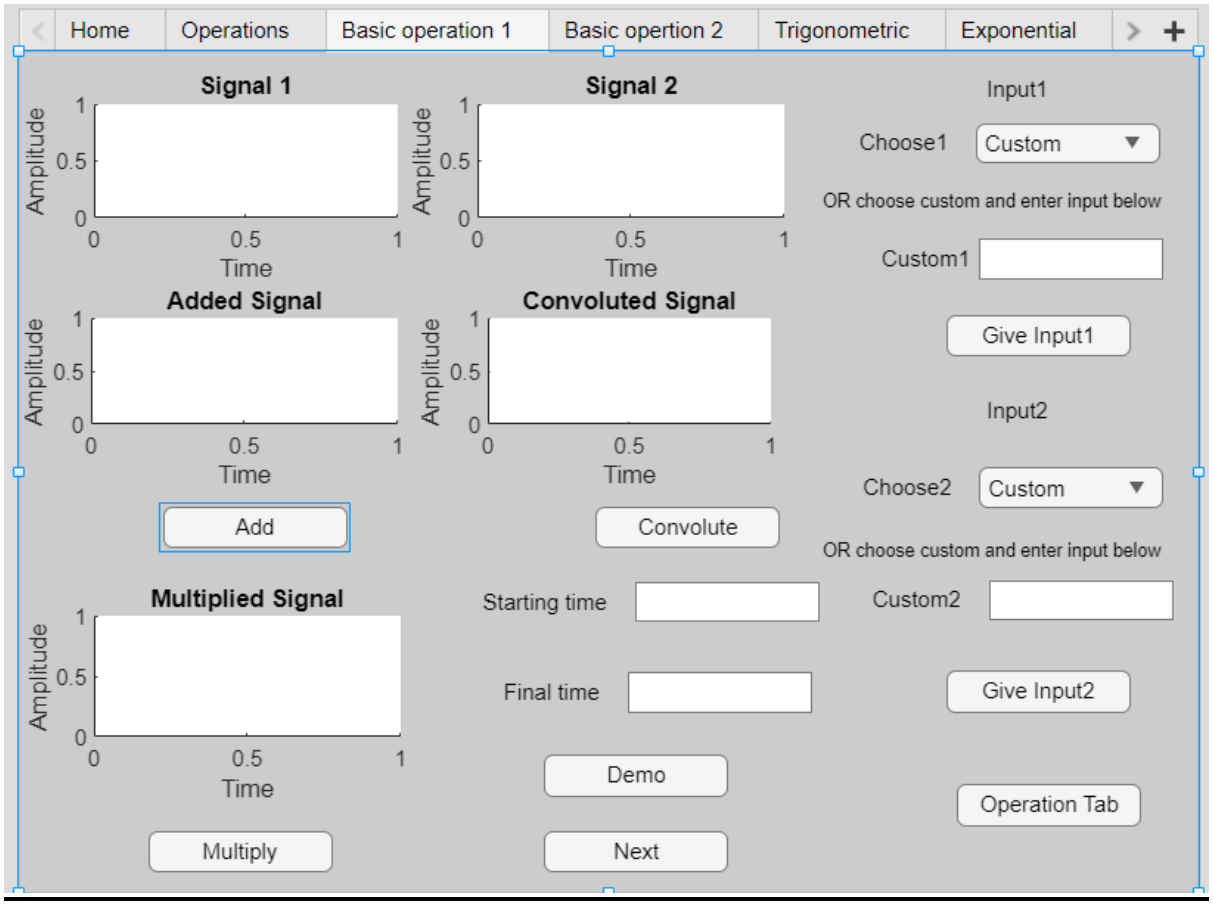
Home:



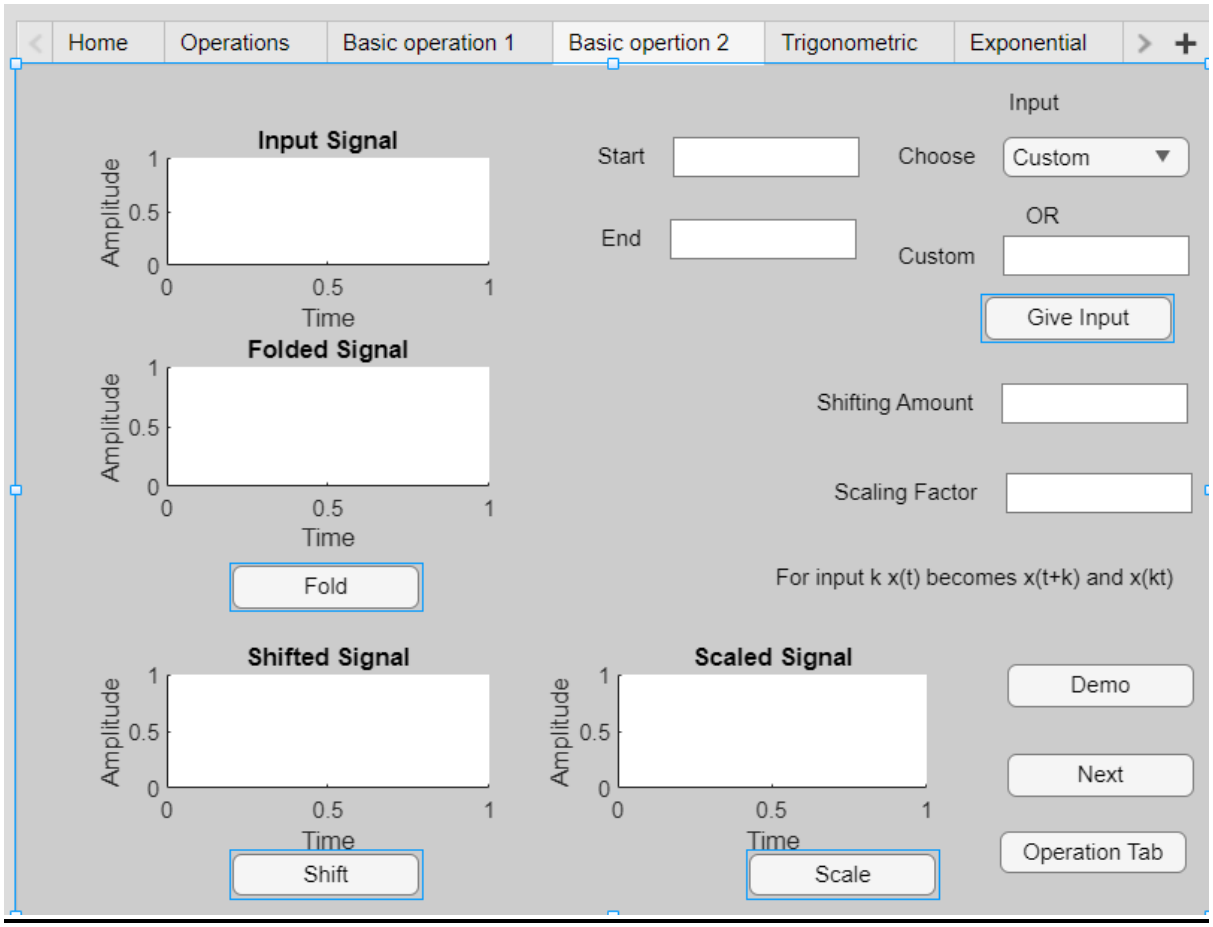
Operations tab:



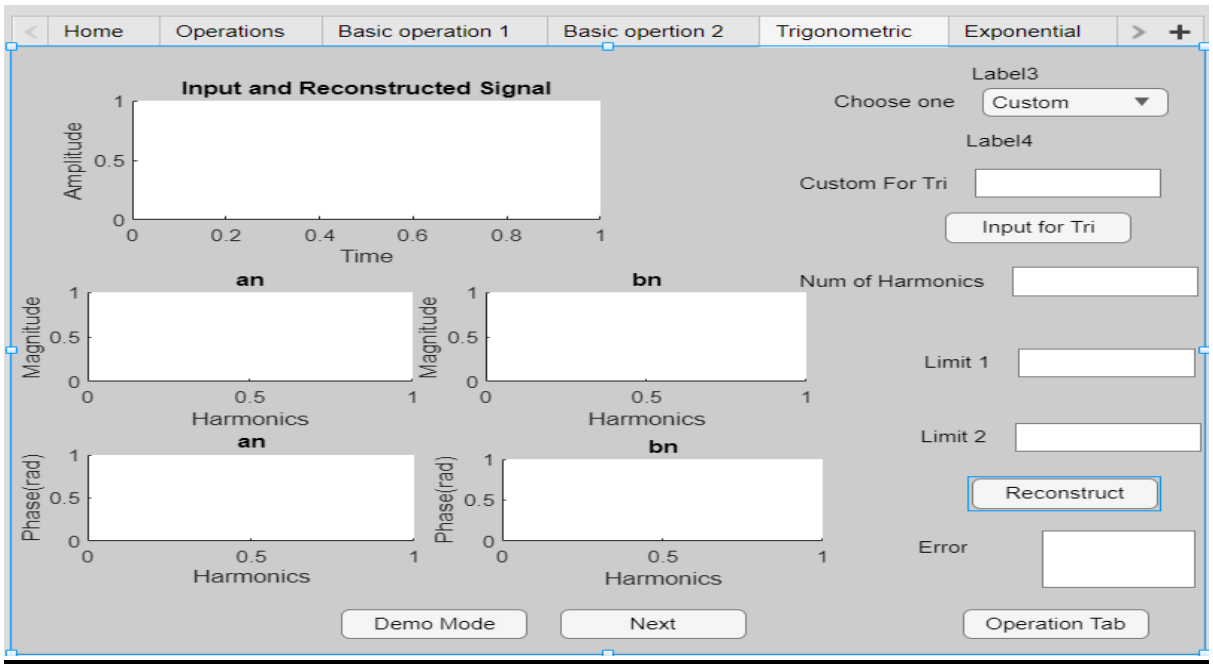
Basic Operations1 Tab



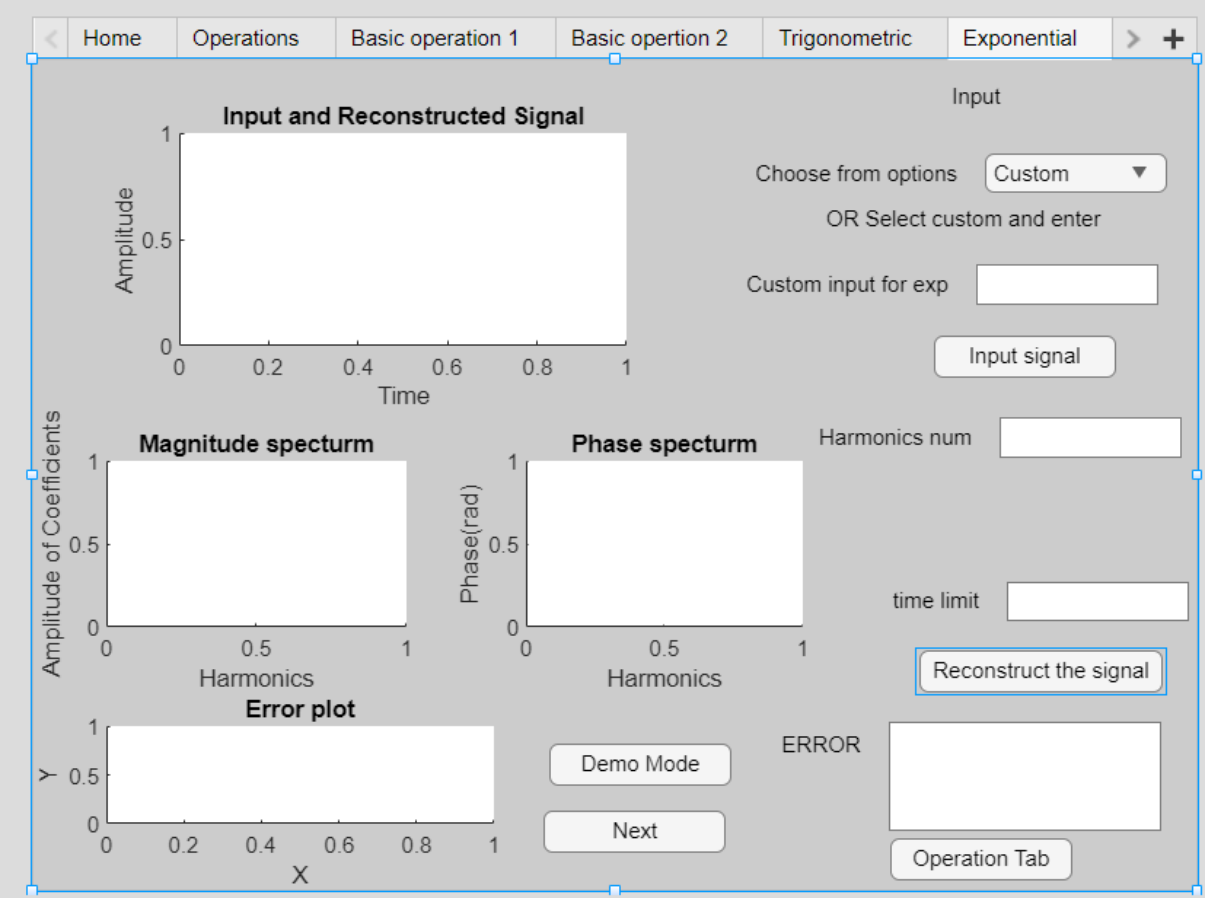
Basic Operations2 tab:



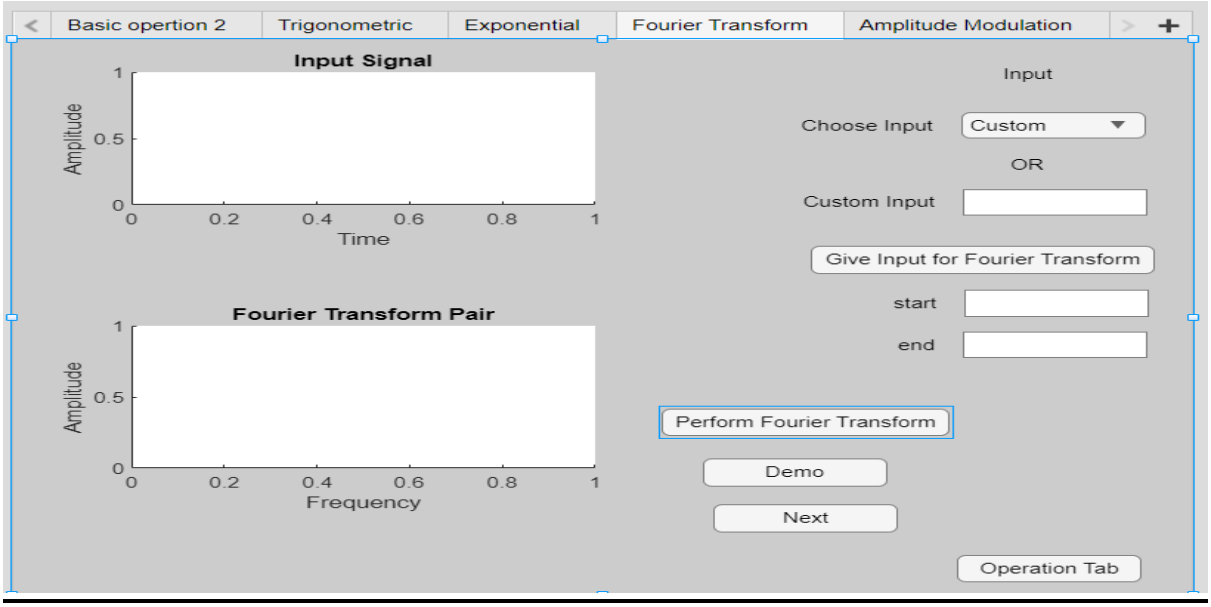
Trigonometric Fourier Series Tab:



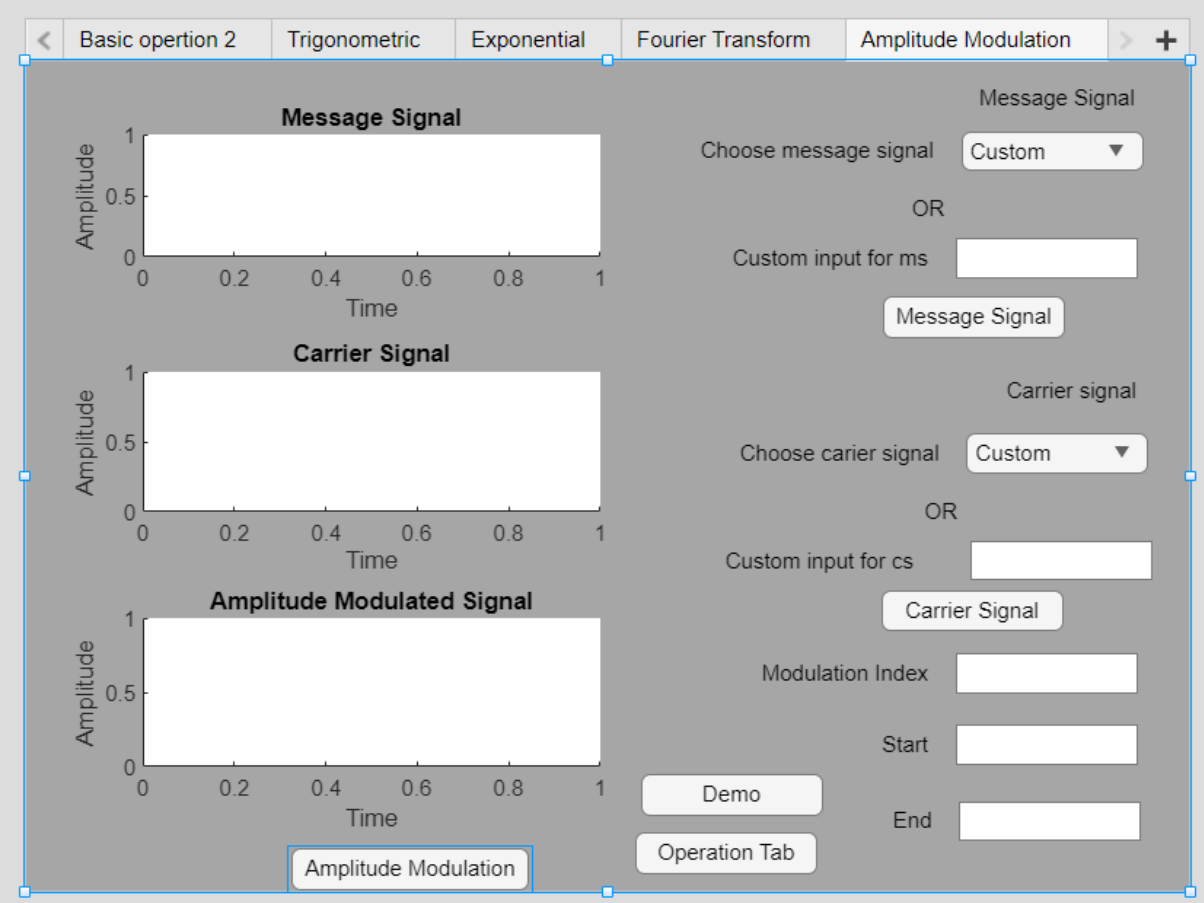
Exponential Fourier Series Tab:



Fourier Transform tab:



Amplitude modulation tab:



3.2. User Manual

- User can choose input from dropdown box or can also select the custom and give any arbitrary function as input. User must be cautious of defining the time range before pressing the input button.
- While shifting and scaling user must specify the amount to be shifted or scaled. Here x(t) will be converted to x(t+k) for input k and to x(kt).
- The output of convolution is shown twice the time limit of input. So some cases where there is no input graph of convoluted signal will be 0.
- Fourier series was implemented and the errors are found by integrating the mismatched part of the input and reconstructed signals.
- For amplitude modulation user should specify modulation index.

4.Main Code

Input: Code for input in various tabs:

```
cla(app.UIAxes13);
        cla(app.UIAxes14);
          t1=str2num(app.StartEditField_2.Value);
         t2=str2num(app.EndEditField_2.Value);

          t = t1:0.01:t2;
          %assignin('base','str',app.InputEditField.Value)
    % Use a switch statement to plot different types of signals

          %assignin('base','str',app.InputEditField.Value)
    % Use a switch statement to plot different types of signals
    switch app.ChoosecariersignalDropDown.Value
        case 'Square Wave'
            app.sht = square(10*t);
            plot(app.UIAxes13, t, app.sht);
        case 'Sine'
            app.sht = sin(10*t);
            plot(app.UIAxes13, t, app.sht);
        case 'Cosine'
            app.sht = cos(10*t);
            plot(app.UIAxes13, t, app.sht);
        case 'Saw tooth'
            app.sht = sawtooth(10*t);
            plot(app.UIAxes13, t, app.sht);
        case 'Rect'
            app.sht = rectpuls(10*t);
            plot(app.UIAxes13, t, app.sht);
        case 'Step'
            app.sht= heaviside(t);
            plot(app.UIAxes13,t,app.sht);
        case 'Ramp'
            app.sht=t.*heaviside(t);
            plot(app.UIAxes13,t,app.sht);
        case 'Delta'
           app.sht=dirac(t);
           idx = app.sht == Inf; % find Inf
           app.sht(idx) = 1;     % set Inf to finite value
           stem(app.UIAxes13, t,app.sht);
        case 'Custom'
           t = t1:0.01:t2;
          f=app.CustominputforcsEditField.Value;
          app.sht=eval(f);
          plot(app.UIAxes13,t,app.sht);
    end
    app.in7=@(t) app.sht;
```

Same algorithm was followed for input in all the cases.Here user can either give custom input or can choose one from the drop down box.

Code for Operations drop down box:

```
% Value changed function: OperationDropDown
            value = app.OperationsDropDown.Value;
            if strcmp(value, 'Basic Operation 1')
                app.TabGroup.SelectedTab = app.Basicoperation1Tab;
                app.DemoButton.Visible="off";
                app.NextButton.Visible="off";
            elseif strcmp(value, 'Basic Operation 2')
                app.TabGroup.SelectedTab = app.Basicopertion2Tab;
                app.DemoButton_2.Visible="off";
                app.NextButton_2.Visible="off";
            elseif strcmp(value, 'Trigonometric Fourier Series')
                app.TabGroup.SelectedTab = app.TrigonometricTab;
                app.DemoModeButton.Visible="off";
                app.NextButton_3.Visible="off";
            elseif strcmp(value,'Exponential Fourier Series')
                app.TabGroup.SelectedTab = app.ExponentialTab;
                app.DemoModeButton_2.Visible="off";
                app.NextButton_4.Visible="off";
            elseif strcmp(value, 'Fourier Transform')
                app.TabGroup.SelectedTab = app.FourierTransformTab;
                app.DemoButton_3.Visible="off";
                app.NextButton_5.Visible="off";
            elseif strcmp(value, 'Amplitude Modularion')
                app.TabGroup.SelectedTab = app.AmplitudeModulationTab;
                app.DemoButton_4.Visible="off";

            end
```

Code for add button:

```
t1=str2num(app.StartingtimeEditField.Value);
        t2=str2num(app.FinaltimeEditField.Value);

        t = t1:0.01:t2;
        z=app.x+app.y;
        plot(app.UIAxes3,t,z);
```

Code for multiply button:

```
t1=str2num(app.StartingtimeEditField.Value);
        t2=str2num(app.FinaltimeEditField.Value);
```

```
t = t1:0.01:t2;
z=app.x.*app.y;
plot(app.UIAxes4,t,z);
```

Code for convolute button

```
t1=str2num(app.StartingtimeEditField.Value);
t2=str2num(app.FinaltimeEditField.Value);
t = t1:0.01:t2;
a=app.x;
b=app.y;
z=conv(a,b);
t_new=linspace(2*t1,2*t2,length(z));
plot(app.UIAxes5,t_new,z);
```

Code for fold button:

```
t1=str2num(app.StartEditField.Value);
t2=str2num(app.EndEditField.Value);
t = t1:0.01:t2;
c=fliplr(app.r);
plot(app.UIAxes7,t,c);
```

Code for shift button:

```
t1=str2num(app.StartEditField.Value);
t2=str2num(app.EndEditField.Value);
t = t1:0.01:t2;
k = app.ShiftingAmountEditField.Value;
p = str2num(k);
t_new = t1+p:0.01:t2; % Calculate the new time range
app.swe = interp1(t, app.r, t_new, 'linear'); % Interpolate the signal onto the new time range
t_shifted = t+p; % Calculate the time range after the shift
app.swet = interp1(t_new, app.swe, t_shifted, 'linear'); % Interpolate the shifted signal onto the original time range
plot(app.UIAxes8, t, app.swet); % Plot the shifted signal
```

Code for scale button:

```
t1=str2num(app.StartEditField.Value);
t2=str2num(app.EndEditField.Value);
t = t1:0.01:t2;
```

```
        k2 = app.ScalingFactorEditField.Value;
        p2 = str2num(k2);
        t_new = t1*p2:0.01:t2*p2; % Calculate the new time range
        app.swee = interp1(t, app.r, t_new, 'linear'); % Interpolate the signal onto the new time
range
        t_scaled = t*p2; % Calculate the time range after the scaling
        app.swett = interp1(t_new, app.swee, t_scaled, 'linear'); % Interpolate the scaled signal
onto the original time range
        plot(app.UIAxes9, t, app.swett); % Plot the scaled signal
```

Code for trigonometric fourier series:

```
cla(app.UIAxes15);
      switch app.ChooseoneDropDown.Value
          case 'Square Wave'



syms t;

% Get user input
N = str2num(app.NumofHarmonicsEditField.Value);
t1 = str2num(app.Limit1EditField.Value);
t2 = str2num(app.Limit2EditField.Value);
T = t2 - t1;
T1 = 2*pi;
w0 = 2*pi/T1;
a = t1:0.01:t2;
b = square(a);
n = 1:N;

% Calculate Fourier series coefficients
a0 = 1/T1*(int(1,t,0,T1/2)+int(-1,t,T1/2,T1));
an = (2/T1)*(int(1*(cos(n*w0*t)), t, 0, T1/2)+int(-1*(cos(n*w0*t)), t, T1/2, T1));
bn = (2/T1)*(int(1*(sin(n*w0*t)), t, 0, T1/2)+int(-1*(sin(n*w0*t)), t, T1/2, T1));

% Calculate Fourier series
Fx = a0 + sum(an.*cos(n*w0*t)) + sum(bn.*sin(n*w0*t));
Fx = vpa(Fx, 4);
c = subs(Fx, t, a);

% Calculate error
error = sum(abs(b - double(c)))/12;

% Display error
app.ErrorTextArea.Value = [num2str(error)];

% Plot the results
plot(app.UIAxes15, a, b);
```

```
hold(app.UIAxes15, 'on');
plot(app.UIAxes15, a, c);
ft_plot.LineWidth = 2;
title(app.UIAxes15, ["The Fourier series up to ", num2str(N), " harmonics is:"]);
legend(app.UIAxes15, ["Given Function", "Fourier"]);

% Plot the coefficients and phase angles
an_values = double(an);
bn_values = double(bn);
an_phases = atan2(-bn_values, an_values);
bn_phases = atan2(an_values, -bn_values);

stem(app.UIAxes16, n, an_values(1:N));
title(app.UIAxes16, "an coefficients");
xlabel(app.UIAxes16, "Harmonic number");
ylabel(app.UIAxes16, "Coefficient value");

stem(app.UIAxes18, n, an_phases(1:N));
title(app.UIAxes18, "an phase angles");
xlabel(app.UIAxes18, "Harmonic number");
ylabel(app.UIAxes18, "Phase angle (rad)");

stem(app.UIAxes17, n, bn_values(1:N));
title(app.UIAxes17, "bn coefficients");
xlabel(app.UIAxes17, "Harmonic number");
ylabel(app.UIAxes17, "Coefficient value");

stem(app.UIAxes19, n, bn_phases(1:N));
title(app.UIAxes19, "bn phase angles");
xlabel(app.UIAxes19, "Harmonic number");
ylabel(app.UIAxes19, "Phase angle (rad)");

        case 'Sine'
            syms t;
            ft = str2sym(app.ftt);
            t1 = str2num(app.Limit1EditField.Value);
            t2 = str2num(app.Limit2EditField.Value);
            N = str2num(app.NumofHarmonicsEditField.Value);
            T = t2 - t1;
            w0 = 2*pi./T;
            n = 1:N;
            a0 = (1/T).*int(ft, t, t1, t2);
            an = (2/T).*int(ft.*cos(n.*w0.*t), t, t1, t2);
            bn = (2/T).*int(ft.*sin(n*w0*t), t, t1, t2);
            Fx = a0 + sum(an.*cos(n.*w0.*t)) + sum(bn.*sin(n.*w0.*t));
            Fx = vpa(Fx, 4);
            %

            err_s = abs(ft - Fx);
            err_fn = matlabFunction(err_s); % Convert to a MATLAB function
```

```
            error = integral(@(t) err_fn(t), t1, t2, 'AbsTol', 1e-12, 'RelTol', 1e-6);


            fplot(app.UIAxes15,ft, [t1, t2],'LineWidth',2);
            hold(app.UIAxes15,'on');
            ft_plot=fplot(app.UIAxes15,Fx, [t1, t2]);
            ft_plot.LineWidth = 2; % increase the line width of Fourier series plot
            title(app.UIAxes15,["The Fourier series up to ", num2str(N), " harmonics is:"]);
            legend(app.UIAxes15, ["Given Function", "Fourier"]); % add legend to show both
plots
            app.ErrorTextArea.Value = [num2str(error)];
             % Calculate the Fourier series coefficients and phase angles for plotting
            an_values = double(an);
            bn_values = double(bn);
            an_phases = atan2(-bn_values, an_values);
            bn_phases = atan2(an_values, -bn_values);


            stem(app.UIAxes16,n, an_values(1:N));
            title(app.UIAxes16,"an coefficients");
            xlabel(app.UIAxes16,"Harmonic number");
            ylabel(app.UIAxes16,"Coefficient value");


            stem(app.UIAxes18,n, an_phases(1:N));
            title(app.UIAxes18,"an phase angles");
            xlabel(app.UIAxes18,"Harmonic number");
            ylabel(app.UIAxes18,"Phase angle (rad)");


            stem(app.UIAxes17,n, bn_values(1:N));
            title(app.UIAxes17,"bn coefficients");
            xlabel(app.UIAxes17,"Harmonic number");
            ylabel(app.UIAxes17,"Coefficient value");


            stem(app.UIAxes19,n, bn_phases(1:N));
            title(app.UIAxes19,"bn phase angles");
            xlabel(app.UIAxes19,"Harmonic number");
            ylabel(app.UIAxes19,"Phase angle (rad)");
        case 'Cosine'
            syms t;
            ft = str2sym(app.ftt);
            t1 = str2num(app.Limit1EditField.Value);
            t2 = str2num(app.Limit2EditField.Value);
            N = str2num(app.NumofHarmonicsEditField.Value);
            T = t2 - t1;
            w0 = 2*pi./T;
            n = 1:N;
            a0 = (1/T).*int(ft, t, t1, t2);
```

```
            an = (2/T).*int(ft.*cos(n.*w0.*t), t, t1, t2);
            bn = (2/T).*int(ft.*sin(n*w0*t), t, t1, t2);
            Fx = a0 + sum(an.*cos(n.*w0.*t)) + sum(bn.*sin(n.*w0.*t));
            Fx = vpa(Fx, 4);
            %

            err_s = abs(ft - Fx);
            err_fn = matlabFunction(err_s); % Convert to a MATLAB function
            error = integral(@(t) err_fn(t), t1, t2, 'AbsTol', 1e-12, 'RelTol', 1e-6);


            fplot(app.UIAxes15,ft, [t1, t2],'LineWidth',2);
            hold(app.UIAxes15,'on');
            ft_plot=fplot(app.UIAxes15,Fx, [t1, t2]);
            ft_plot.LineWidth = 2; % increase the line width of Fourier series plot
            title(app.UIAxes15,["The Fourier series up to ", num2str(N), " harmonics is:"]);
            legend(app.UIAxes15, ["Given Function", "Fourier"]); % add legend to show both
plots
            app.ErrorTextArea.Value = [num2str(error)];
             % Calculate the Fourier series coefficients and phase angles for plotting
            an_values = double(an);
            bn_values = double(bn);
            an_phases = atan2(-bn_values, an_values);
            bn_phases = atan2(an_values, -bn_values);


            stem(app.UIAxes16,n, an_values(1:N));
            title(app.UIAxes16,"an coefficients");
            xlabel(app.UIAxes16,"Harmonic number");
            ylabel(app.UIAxes16,"Coefficient value");


            stem(app.UIAxes18,n, an_phases(1:N));
            title(app.UIAxes18,"an phase angles");
            xlabel(app.UIAxes18,"Harmonic number");
            ylabel(app.UIAxes18,"Phase angle (rad)");


            stem(app.UIAxes17,n, bn_values(1:N));
            title(app.UIAxes17,"bn coefficients");
            xlabel(app.UIAxes17,"Harmonic number");
            ylabel(app.UIAxes17,"Coefficient value");


            stem(app.UIAxes19,n, bn_phases(1:N));
            title(app.UIAxes19,"bn phase angles");
            xlabel(app.UIAxes19,"Harmonic number");
            ylabel(app.UIAxes19,"Phase angle (rad)");
        case 'Custom'
            syms t;
```

```
ft = str2sym(app.ftt);
t1 = str2num(app.Limit1EditField.Value);
t2 = str2num(app.Limit2EditField.Value);
N = str2num(app.NumofHarmonicsEditField.Value);
T = t2 - t1;
w0 = 2*pi./T;
n = 1:N;
a0 = (1/T).*int(ft, t, t1, t2);
an = (2/T).*int(ft.*cos(n.*w0.*t), t, t1, t2);
bn = (2/T).*int(ft.*sin(n*w0*t), t, t1, t2);
Fx = a0 + sum(an.*cos(n.*w0.*t)) + sum(bn.*sin(n.*w0.*t));
Fx = vpa(Fx, 4);
%

err_s = abs(ft - Fx);
err_fn = matlabFunction(err_s); % Convert to a MATLAB function
error = integral(@(t) err_fn(t), t1, t2, 'AbsTol', 1e-12, 'RelTol', 1e-6);


fplot(app.UIAxes15,ft, [t1, t2],'LineWidth',2);
hold(app.UIAxes15,'on');
ft_plot=fplot(app.UIAxes15,Fx, [t1, t2]);
ft_plot.LineWidth = 2; % increase the line width of Fourier series plot
title(app.UIAxes15,["The Fourier series up to ", num2str(N), " harmonics is:"]);
legend(app.UIAxes15, ["Given Function", "Fourier"]); % add legend to show both
plots

app.ErrorTextArea.Value = [num2str(error)];
 % Calculate the Fourier series coefficients and phase angles for plotting
an_values = double(an);
bn_values = double(bn);
an_phases = atan2(-bn_values, an_values);
bn_phases = atan2(an_values, -bn_values);


stem(app.UIAxes16,n, an_values(1:N));
title(app.UIAxes16,"an coefficients");
xlabel(app.UIAxes16,"Harmonic number");
ylabel(app.UIAxes16,"Coefficient value");


stem(app.UIAxes18,n, an_phases(1:N));
title(app.UIAxes18,"an phase angles");
xlabel(app.UIAxes18,"Harmonic number");
ylabel(app.UIAxes18,"Phase angle (rad)");


stem(app.UIAxes17,n, bn_values(1:N));
title(app.UIAxes17,"bn coefficients");
xlabel(app.UIAxes17,"Harmonic number");
ylabel(app.UIAxes17,"Coefficient value");
```

```
        stem(app.UIAxes19,n, bn_phases(1:N));
        title(app.UIAxes19,"bn phase angles");
        xlabel(app.UIAxes19,"Harmonic number");
        ylabel(app.UIAxes19,"Phase angle (rad)");
    end
```

Code for exponential fourier series:

```
cla(app.UIAxes20);

        % Define the time domain
        t1=0;
        t2=str2num(app.timelimitEditField.Value);
        N=str2num(app.HarmonicsnumEditField.Value);
    t = -20:0.01:20;
    t_values=linspace(t1,t2,1000);

% Define the input function
switch app.ChoosefromoptionsDropDown.Value
   case 'Square Wave'
    d = t1:0.01:t2;
    exact_values=square(d);
    T = 2*pi;
    n = -N:N; % modify n to include zero frequency

    w0 = 2*pi/T;

    % Calculate Fourier series coefficients
    syms t;

% Calculate the coefficients

    n = -N:N; % modify n to include zero frequency
    cn = zeros(1,length(n));
    for i = 1:length(n)
        cn(i) = (1/T)*(int(exp(-1j*n(i)*w0*t), t, 0, T/2)+int(-1*exp(-1j*n(i)*w0*t), t, T/2, T));
    end

    % Calculate Fourier series
    expar=exp(1j*n*w0*t); % modify formula to include zero frequency
    Fx=sum(cn.*expar);

    % Plot exact and approximated graphs


    approx_values = double(subs(Fx, d));
```

```
        plot(app.UIAxes20, d, exact_values, 'LineWidth', 2);
        hold (app.UIAxes20,'on');
        plot(app.UIAxes20, d, approx_values, 'LineWidth', 2);
        legend(app.UIAxes20,["Exact", "Approximated"]);
        title(app.UIAxes20,['Exponential Fourier series up to ', num2str(N), ' harmonics']);

        err_s=abs(exact_values-approx_values);
        plot(app.UIAxes23,d,err_s);
        error=sum(err_s)/30;
        app.ERRORTextArea.Value=[num2str(error)];

        % Plot Fourier series coefficients
        cn_values = double(cn);
        stem(app.UIAxes21, n, abs(cn_values));
        % Plot phase of Fourier series coefficients

        stem(app.UIAxes22, n, angle(cn_values));
    case 'Sine'
        ft = @(t) sin(t);
        T = t2 - t1;
n = -N:N; % modify n to include zero frequency
w0 = 2*pi/T;

% Calculate Fourier series coefficients
cn = fouriercoeff(ft, T, N);

% Calculate Fourier series
expar=exp(1j*n*w0*t); % modify formula to include zero frequency
Fx=sum(cn.*expar);

% Evaluate error numerically
err_s = abs(ft - Fx);
err= subs(err_s, t, t_values);
plot(app.UIAxes23,t_values,err);
err_fn = matlabFunction(err_s); % Convert to a MATLAB function
error = integral(@(t) err_fn(t), t1, t2, 'AbsTol', 1e-12, 'RelTol', 1e-6);
app.ERRORTextArea.Value=[num2str(error)];


% Plot exact and approximated graphs

exact_values = double(subs(ft, t_values));
approx_values = double(subs(Fx, t_values));

plot(app.UIAxes20, t_values, exact_values, 'LineWidth', 2);
hold (app.UIAxes20,'on');
plot(app.UIAxes20, t_values, approx_values, 'LineWidth', 2);
%  legend(app.UIAxes15, ["Given Function", "Fourier"]);
legend(app.UIAxes20,["Exact", "Approximated"]);
title(app.UIAxes20,['Exponential Fourier series up to ', num2str(N), ' harmonics']);
```

```
% Plot Fourier series coefficients
cn_values = double(cn);
stem(app.UIAxes21, n, abs(cn_values));

% Plot phase of Fourier series coefficients

stem(app.UIAxes22, n, angle(cn_values));
   case 'Cosine'
       ft = @(t) cos(t);
       T = t2 - t1;
n = -N:N; % modify n to include zero frequency
w0 = 2*pi/T;

% Calculate Fourier series coefficients
cn = fouriercoeff(ft, T, N);

% Calculate Fourier series
expar=exp(1j*n*w0*t); % modify formula to include zero frequency
Fx=sum(cn.*expar);

% Evaluate error numerically
err_s = abs(ft - Fx);
err_fn = matlabFunction(err_s); % Convert to a MATLAB function
error = integral(@(t) err_fn(t), t1, t2, 'AbsTol', 1e-12, 'RelTol', 1e-6);
app.ERRORTextArea.Value=[num2str(error)];


% Plot exact and approximated graphs

err= subs(err_s, t, t_values);
plot(app.UIAxes23,t_values,err);
exact_values = double(subs(ft, t_values));
approx_values = double(subs(Fx, t_values));

plot(app.UIAxes20, t_values, exact_values, 'LineWidth', 2);
hold (app.UIAxes20,'on');
plot(app.UIAxes20, t_values, approx_values, 'LineWidth', 2);
legend(app.UIAxes20,["Exact", "Approximated"]);
title(app.UIAxes20,['Exponential Fourier series up to ', num2str(N), ' harmonics']);



% Plot Fourier series coefficients
cn_values = double(cn);
stem(app.UIAxes21, n, abs(cn_values));

% Plot phase of Fourier series coefficients
```

```
stem(app.UIAxes22, n, angle(cn_values));
   case 'Custom'
       ft = @(t) (app.CustominputforexpEditField.Value);

T = t2 - t1;
n = -N:N; % modify n to include zero frequency
w0 = 2*pi/T;

% Calculate Fourier series coefficients
cn = fouriercoeff(ft, T, N);

% Calculate Fourier series
expar=exp(1j*n*w0*t); % modify formula to include zero frequency
Fx=sum(cn.*expar);

% Evaluate error numerically
err_s = abs(ft - Fx);
err= subs(err_s, t, t_values);
plot(app.UIAxes23,t_values,err);
err_fn = matlabFunction(err_s); % Convert to a MATLAB function
error = integral(@(t) err_fn(t), t1, t2, 'AbsTol', 1e-12, 'RelTol', 1e-6);
app.ERRORTextArea.Value=[num2str(error)];


% Plot exact and approximated graphs

exact_values = double(subs(ft, t_values));
approx_values = double(subs(Fx, t_values));

plot(app.UIAxes20, t_values, exact_values, 'LineWidth', 2);
hold (app.UIAxes20,'on');
plot(app.UIAxes20, t_values, approx_values, 'LineWidth', 2);
legend(app.UIAxes20,["Exact", "Approximated"]);
title(app.UIAxes20,['Exponential Fourier series up to ', num2str(N), ' harmonics']);



% Plot Fourier series coefficients
cn_values = double(cn);
stem(app.UIAxes21, n, abs(cn_values));

% Plot phase of Fourier series coefficients

stem(app.UIAxes22, n, angle(cn_values));
end

function cn = fouriercoeff(f, T, N)
 % Define symbolic variable t
syms t;
```

```
% Calculate the coefficients
w0 = 2*pi/T;
n = -N:N; % modify n to include zero frequency
cn = zeros(1,length(n));
for i = 1:length(n)
    cn(i) = (1/T)*int(f*exp(-1j*n(i)*w0*t), t, 0, T);
end
end

        end
```

Code for Fourier transform:

```
t1=str2num(app.startEditField.Value);
          t2=str2num(app.endEditField.Value);
          t=t1:0.01:t2;
          X = fft(app.s);
          Fs = 1/(t(2)-t(1));
          fvec = linspace(-Fs/2,Fs/2,length(X));
          plot(app.UIAxes11,fvec,fftshift(abs(X)));
```

Code for amplitude modulation:

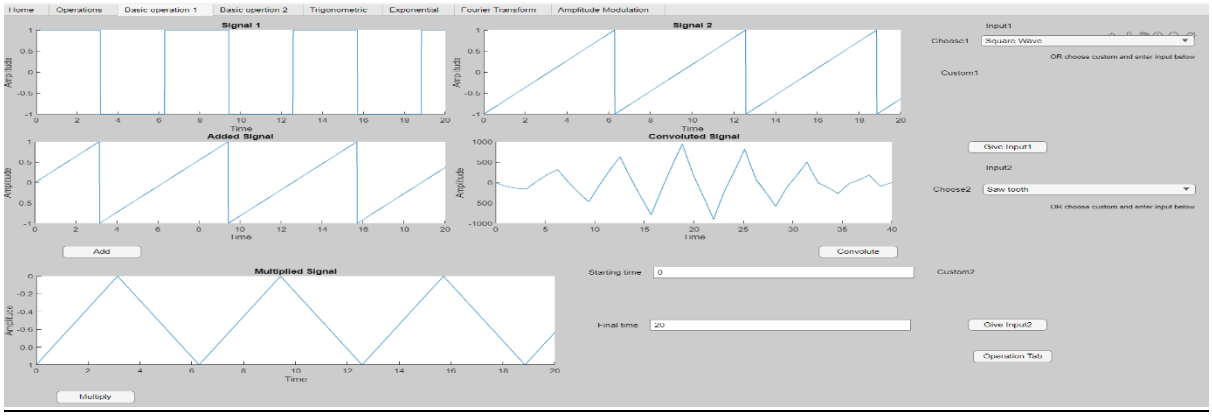
```
t1=str2num(app.StartEditField_2.Value);
          t2=str2num(app.EndEditField_2.Value);

           t = t1:0.01:t2;
         who=app.ModulationIndexEditField.Value;
         app.mui=str2num(who);
         c=(1+app.mui.*app.sh).*app.sht;
         plot(app.UIAxes14,t,c);
```

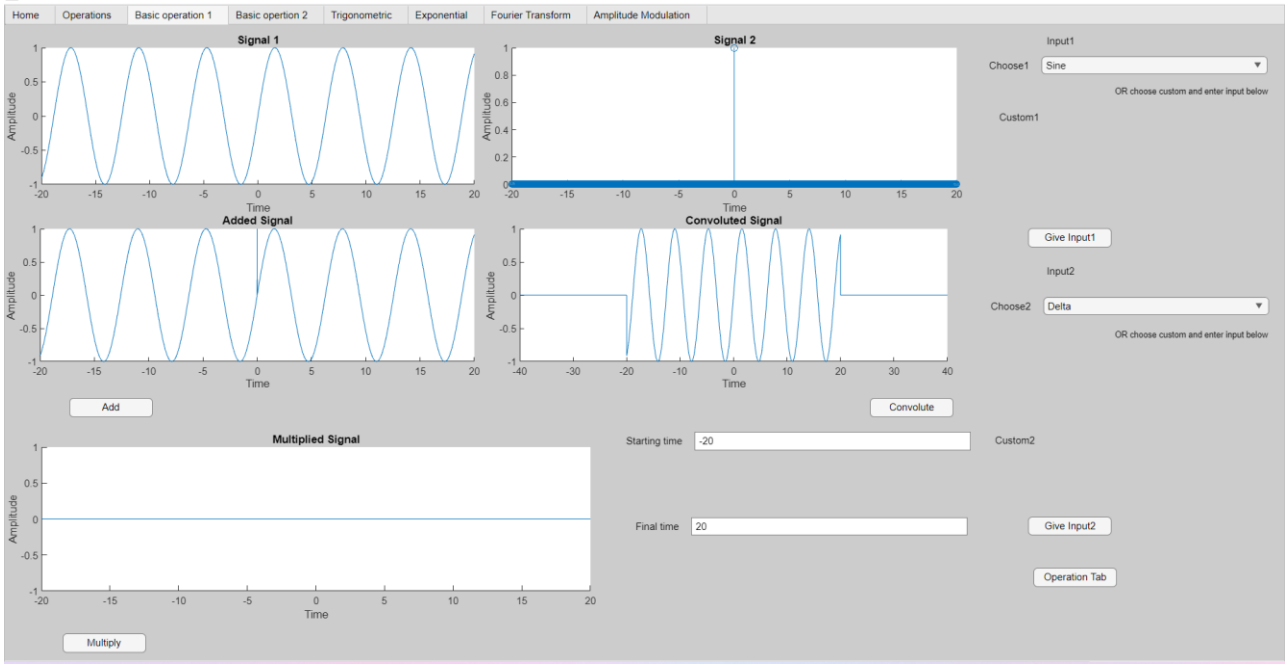
5.Test Cases

Basic operation1:

Example-1: Square and Sawtooth as input

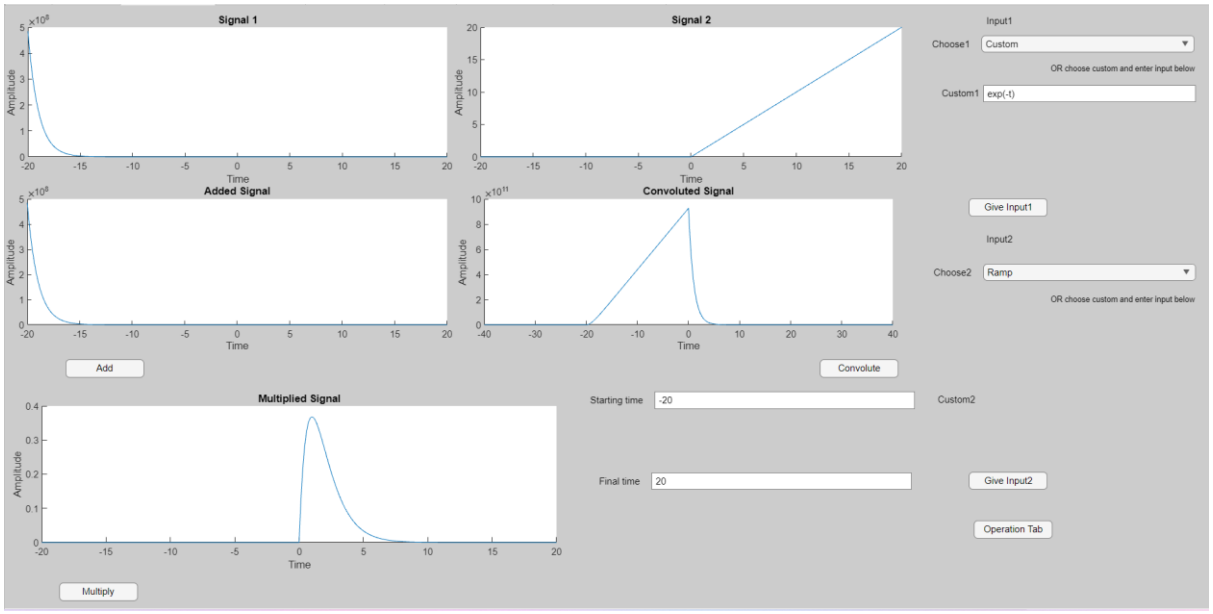


Example-2: Sine and delta as input



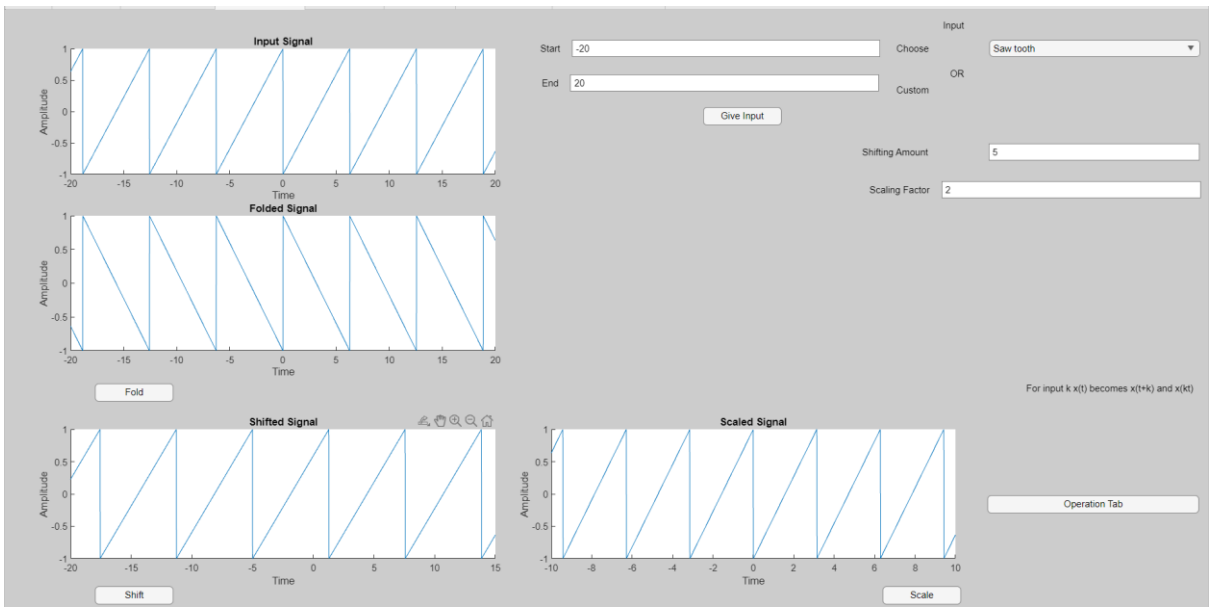
Example-3:

Ramp and custom

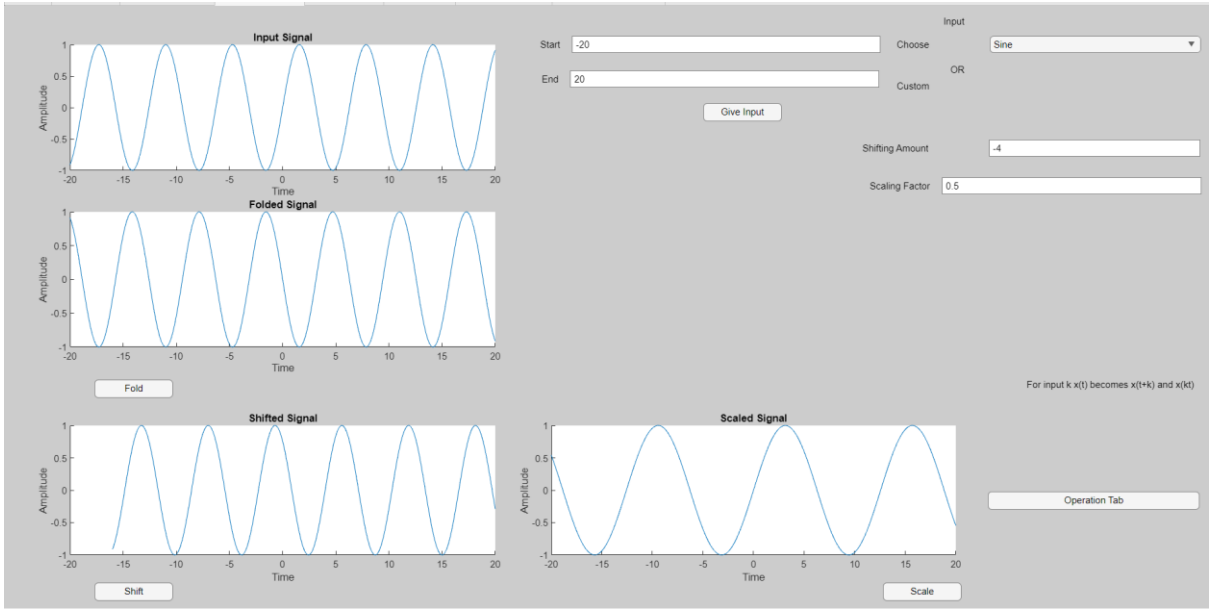


Basic operation2

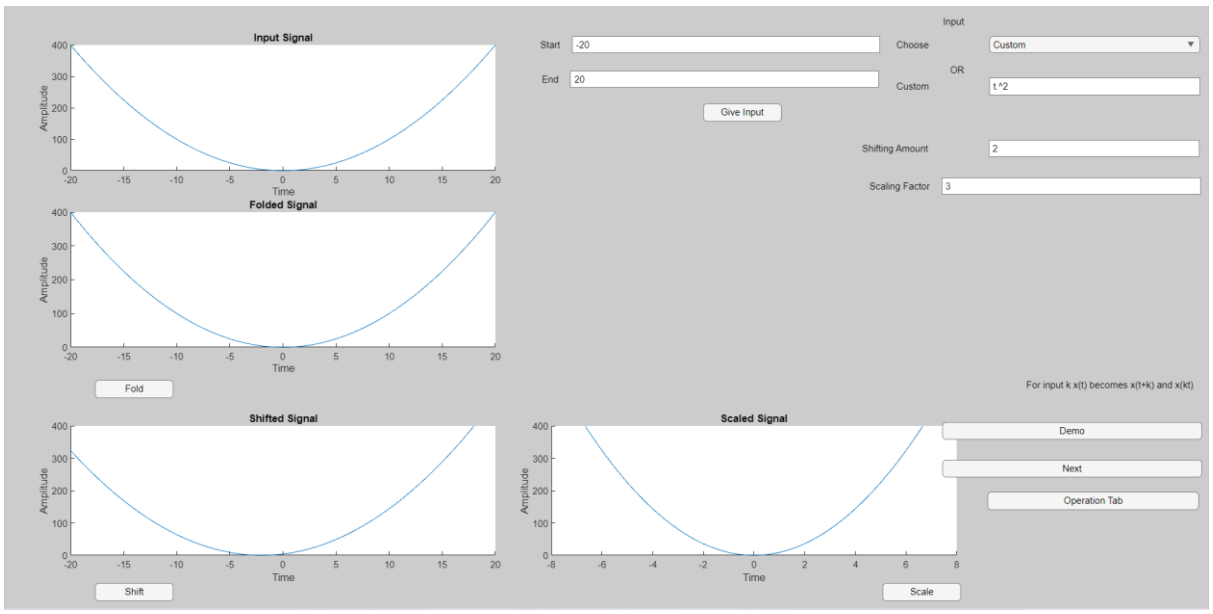
Example-4: Sawtooth



Example-5:Sine

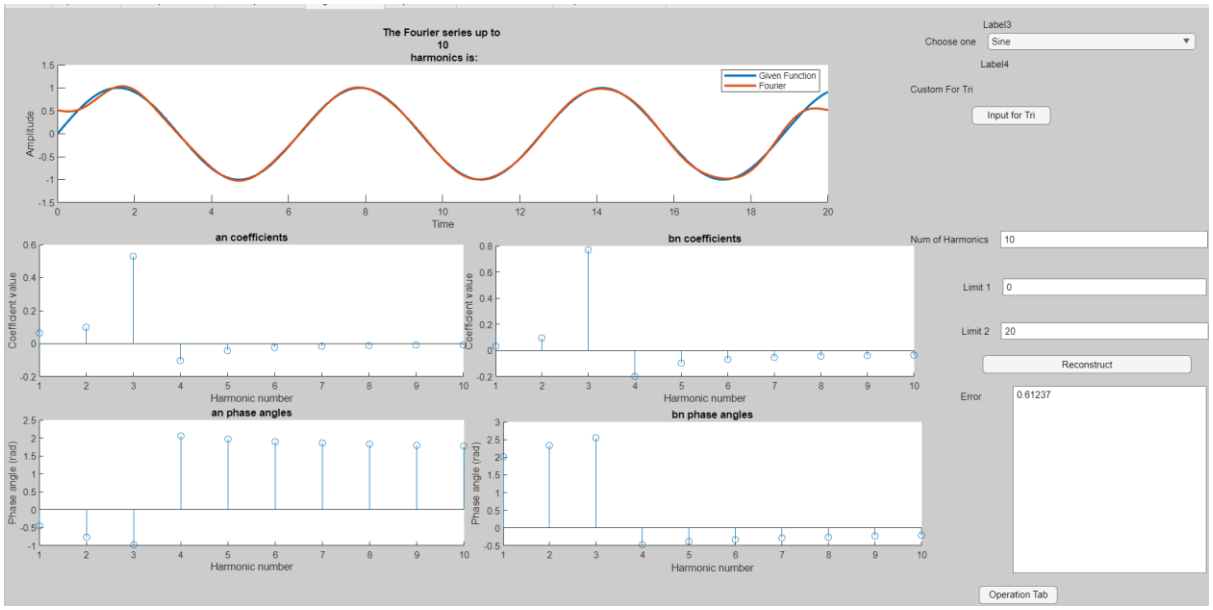


Example-6:

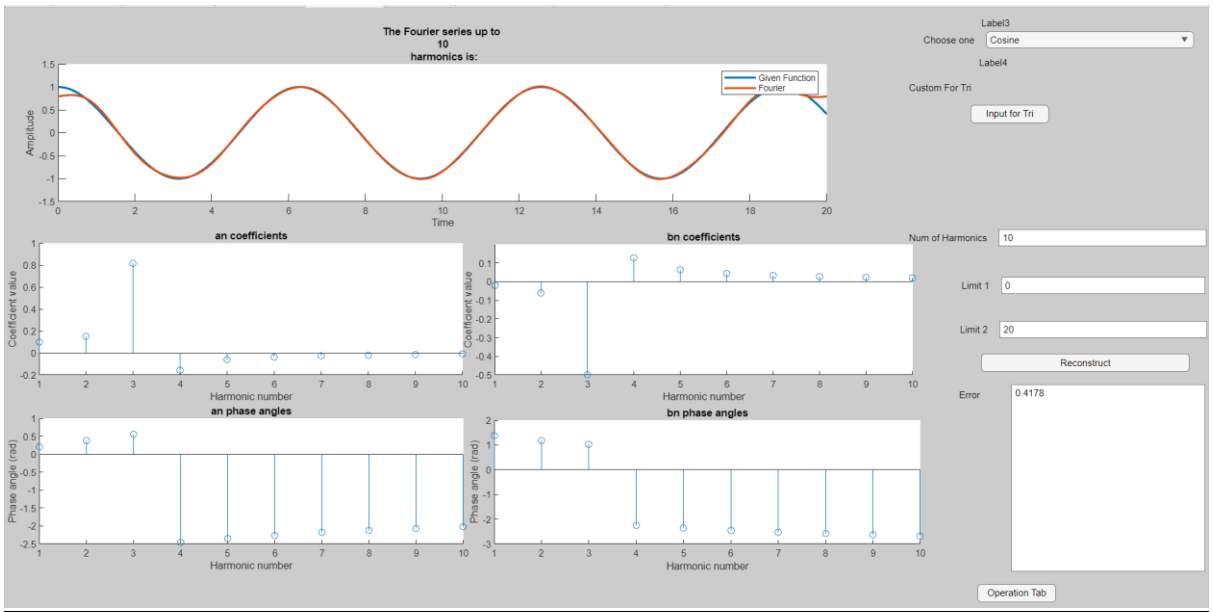


Trigonometric Fourier series:

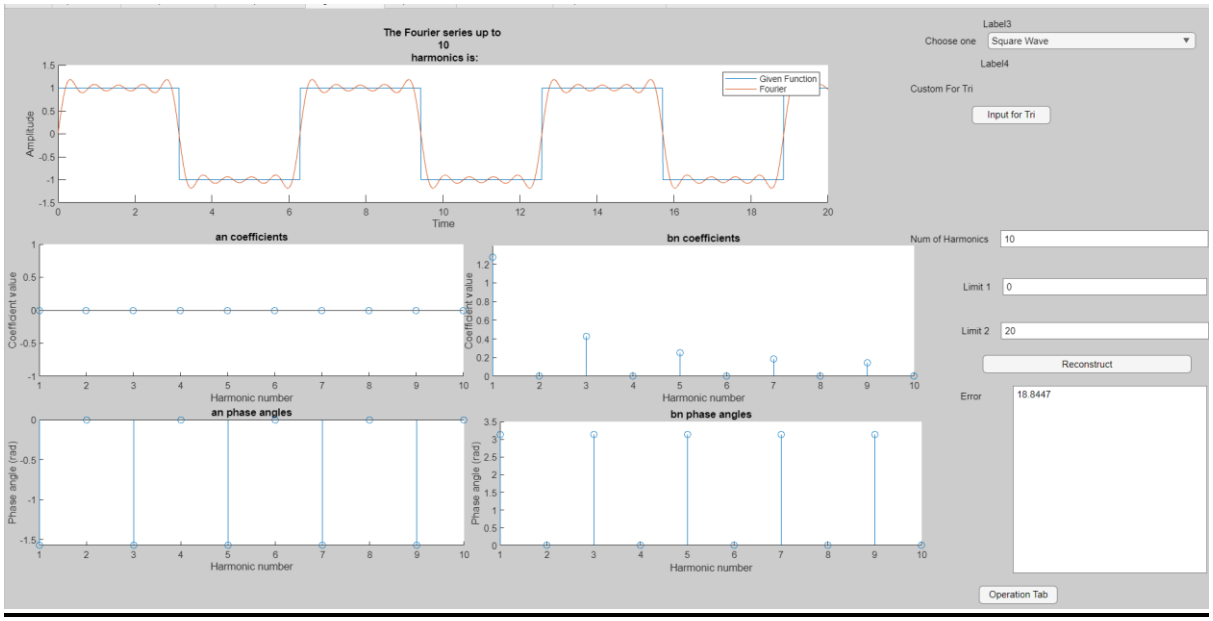
Example-7:Sine



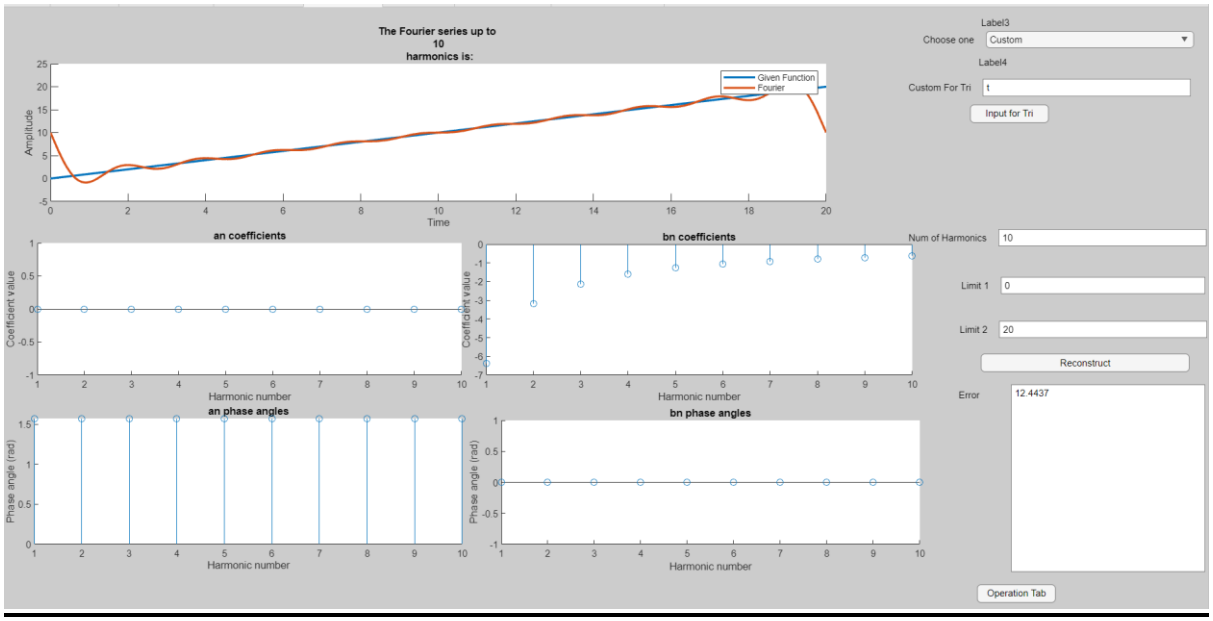
Example-8: Cosine



Example-9: Square

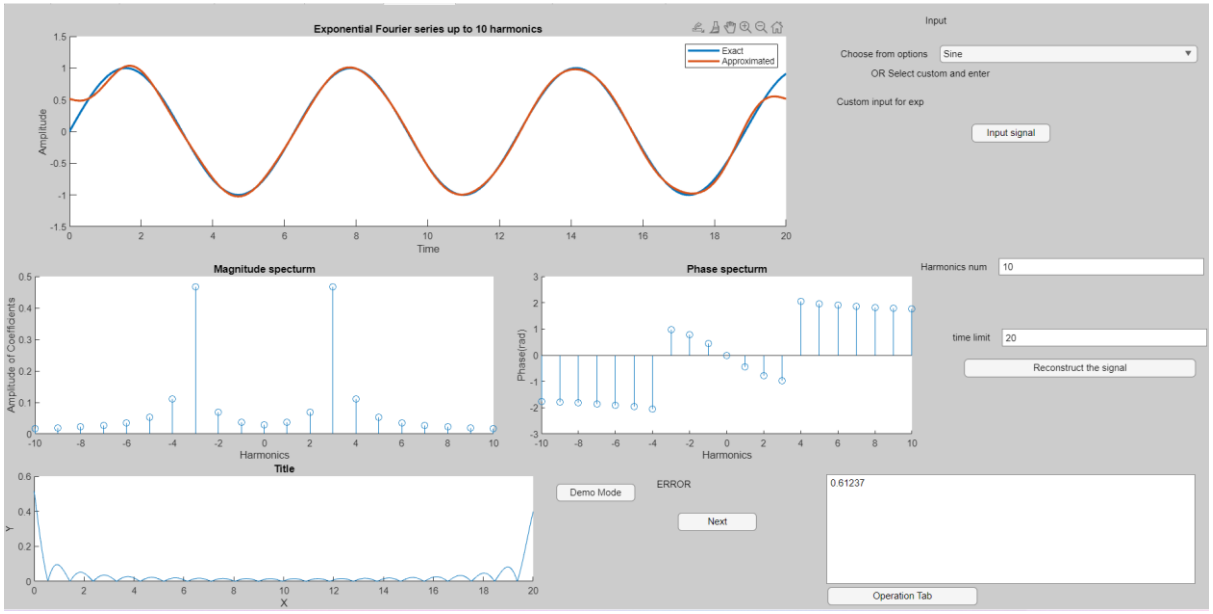


Example-10: Custom input

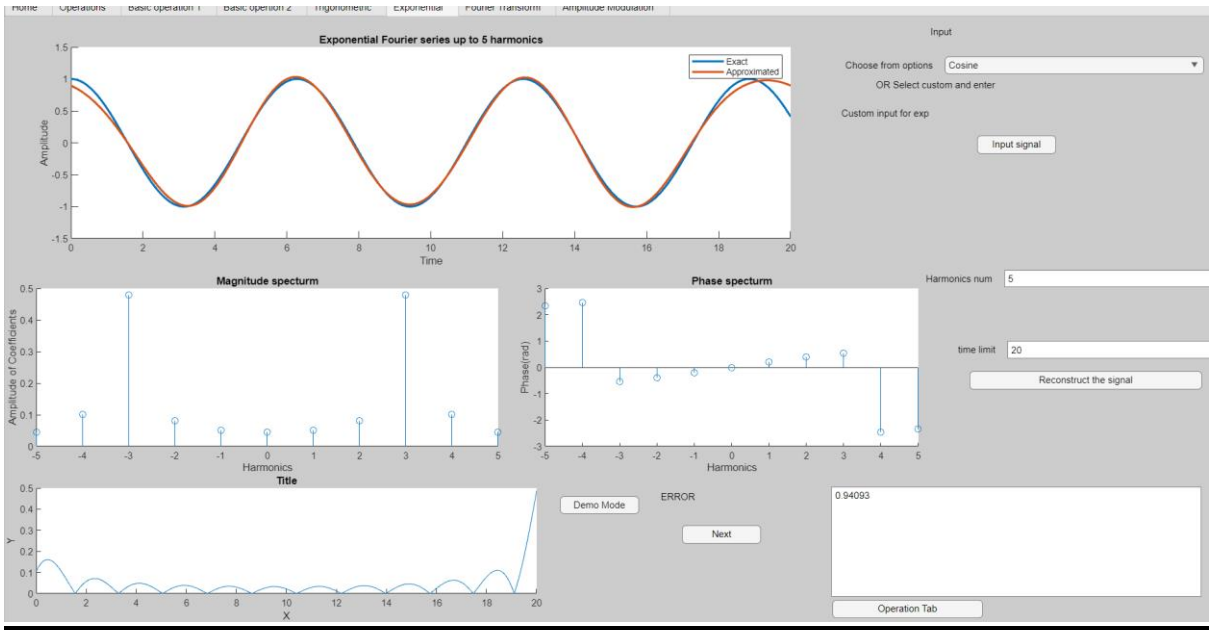


Exponential Fourier Series

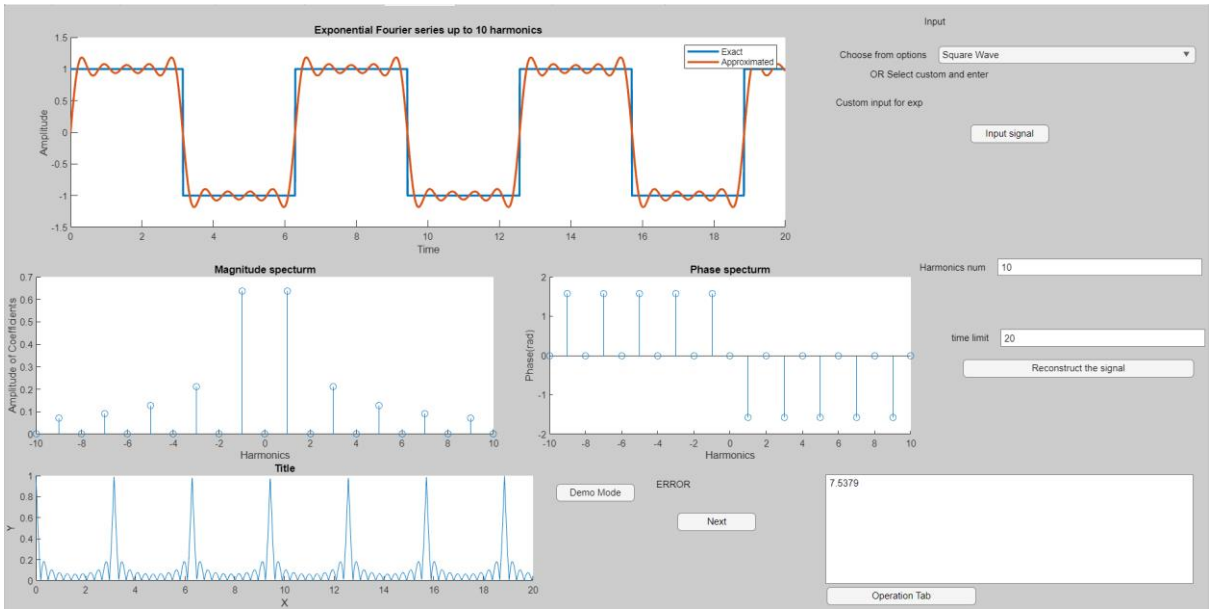
Example-11:



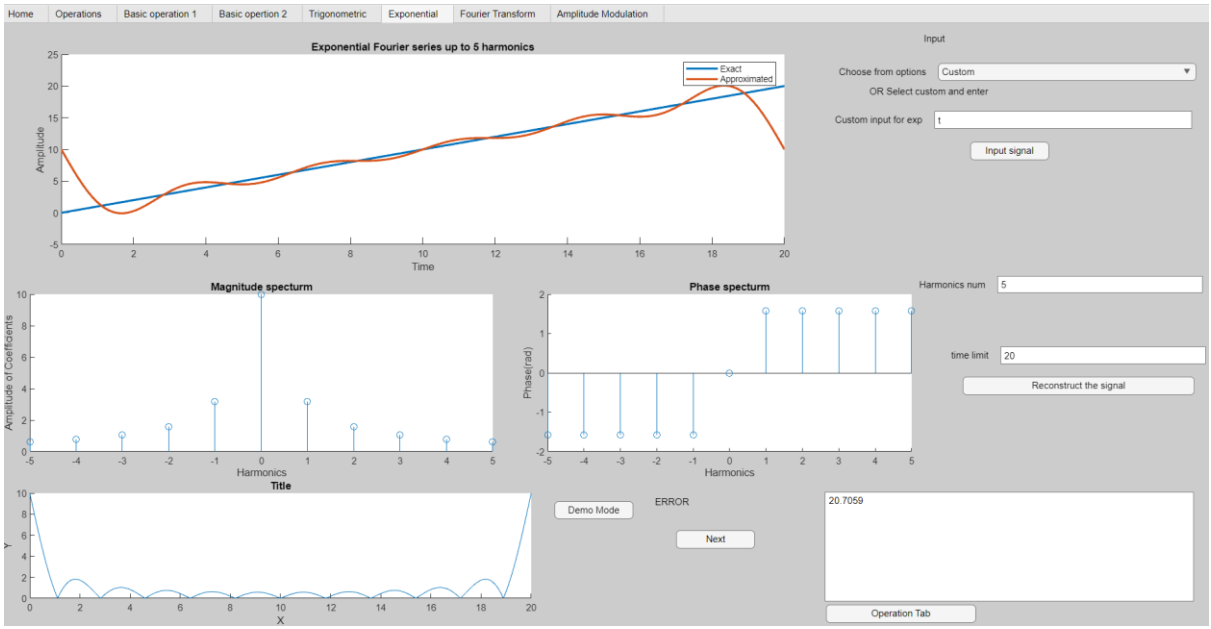
Example-12:



Example-13:

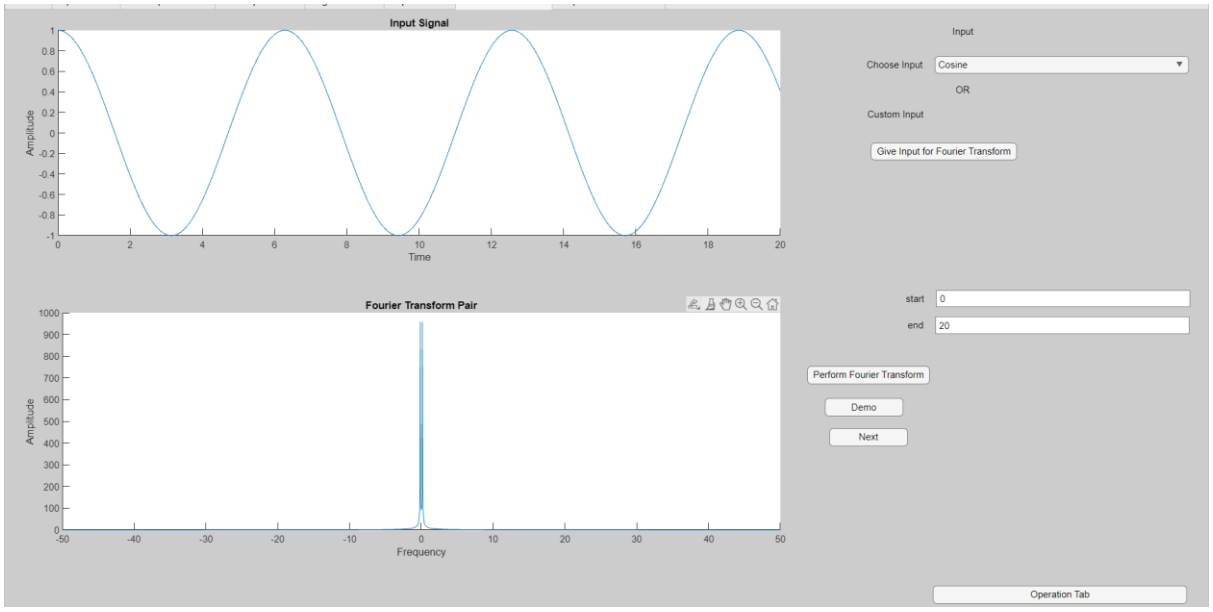


Example-14:

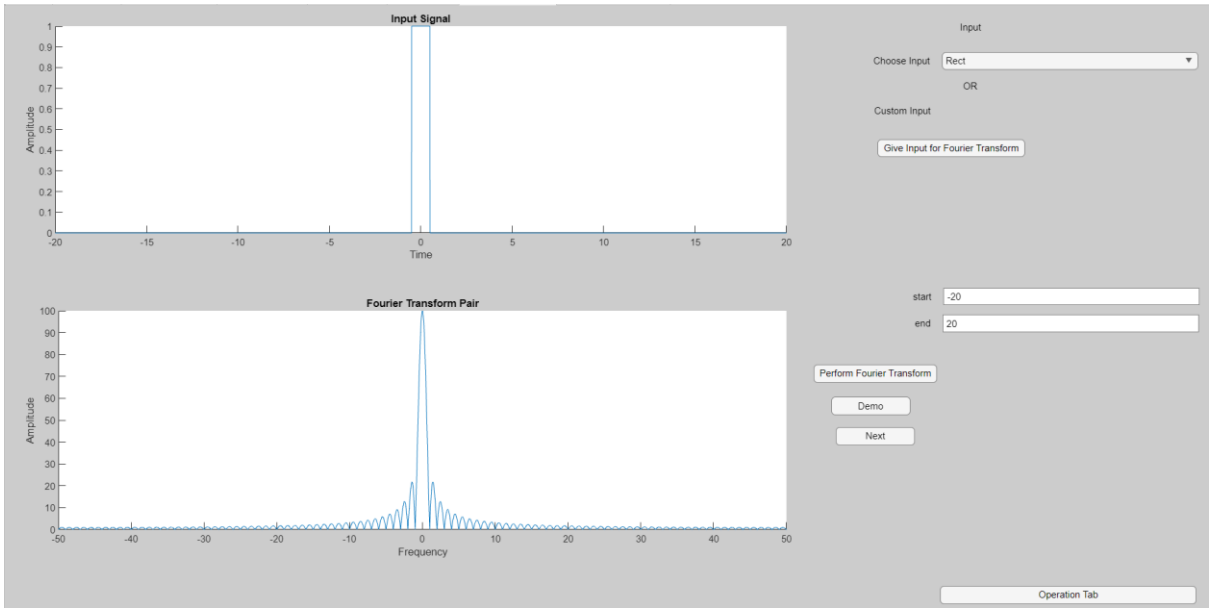


Fourier Transform:

Example-15:

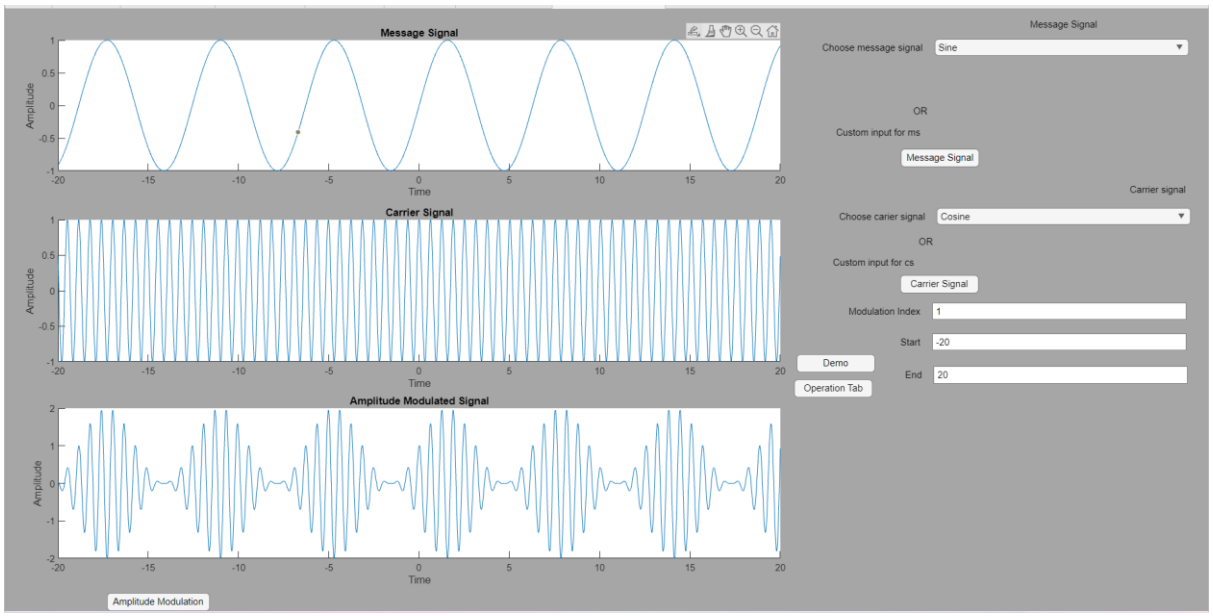


Example-16:



Amplitude modulation:

Example-17:



6.Applications

- A periodic signal can be broken down via Fourier series analysis into a sum of sine and cosine waves with various frequencies and amplitudes. The frequency components of the signal can be examined using this procedure, which is helpful in a wide range of applications, including power systems, audio processing, and communication systems.

- An analysis of a non-periodic signal's frequency components can be done using the Fourier transform. By converting the signal from the time domain to the frequency domain using the Fourier transform, we can see the frequency content of the signal. Several applications, including image processing, medical signal processing, and control systems, can benefit from this approach.

- Amplitude Modulation (AM) is a method for combining a low-frequency message signal with a high-frequency carrier signal. Radio transmission, in which the message signal stands in for the audio content and the carrier signal for the radio frequency, can make advantage of this technique. The demodulated signal is then extracted from the modulated signal by the receiver after the modulated signal has been sent across the airwaves to it.

- Convolution: A digital filter can be applied to a signal via convolution. Many different applications, including control systems, picture processing, and audio processing, use digital filters. Convolution can also be used to model how a system would react to various inputs, which is helpful for system identification and control systems.

- Multiplication: A signal can be modulated by another signal via multiplication. Many applications, such as frequency mixing, which involves multiplying two signals with different frequencies to create a signal with a new frequency, use this technique.

- Changing the amplitude and timing of a signal can be done via scaling and shifting. This technique can be used to change the gain or delay of a signal as well as the brightness or contrast of an image in a variety of applications, including audio processing, image processing, and control systems.

7.Shortcomings

- Computational Complexity: For big signals in particular, several processes, such as the Fourier transform and convolution, can be computationally demanding. Due to memory or processor limitations, the program might not be able to handle strong signals, which could cause errors or crashes.

- Signal noise: Signals are frequently distorted by noise in real-world applications, which can impair the analysis's precision. Analysis problems could result from the app's inability to discriminate between signal and noise components.

- Signal aliasing: The Nyquist-Shannon sampling theorem stipulates that the sampling rate in signal processing applications should be at least twice the highest frequency component of the signal. A low sampling rate might cause aliasing, which will distort the signal and cause analysis errors.

- Limited Signal Type Support: The software could only be able to handle a small selection of signal kinds, such as periodic, continuous-time, and discrete-time signals. Other signal types, such stochastic or non-linear signals, might not be supported, which could result in mistakes or wrong analysis.

- Problems caused by erroneous user input: The app may also experience errors due to inaccurate user input. For instance, the analysis may be inaccurate or invalid if the user specifies improper parameters for an operation, such as the sample rate or frequency range.

8. Conclusion

- In conclusion, the signal analysis software is a helpful tool for processing signals in a range of applications, including control systems, audio processing, and communication systems. Users can extract useful information from signals using the app's various operations, which include Fourier series analysis, Fourier transform, amplitude modulation, convolution, multiplication, and scaling and shifting.

- The signal analysis app does, however, have some drawbacks and restrictions, including processing complexity, signal noise, signal aliasing, a small number of signal kinds, and user input errors. Users must be aware of these restrictions in order to utilize the app effectively for their own needs.

- The signal analysis app has the potential to enhance signal processing and analysis in many sectors despite these drawbacks. The software can be upgraded and enhanced to meet changing user needs as technology develops and new signal processing methods are created. Overall, the signal analysis app is a useful resource for anyone dealing with signals and has the potential to lead to new discoveries and signal processing applications.